

RELATÓRIO TÉCNICO – TRABALHO FINAL FTC

Disciplina: Fundamentos Teóricos da Computação

Professor: Júlio César da Silva

Data: 09/06/2026

Pedro Lopes Rodrigues & Maria Luiza Bertolino Matos

Belo Horizonte

2026

1. INTRODUÇÃO

Este trabalho tem como objetivo implementar três máquinas abstratas fundamentais da Teoria da Computação utilizando a linguagem C# (.NET 6): Autômato Finito Determinístico (AFD), Autômato de Pilha (AP) com aceitação por pilha vazia e Máquina de Turing (MT). Cada máquina foi projetada para reconhecer uma linguagem específica, demonstrando na prática os diferentes níveis da Hierarquia de Chomsky e os limites computacionais de cada modelo. O trabalho também explora desafios adicionais, como a carga dinâmica de AFD via arquivo JSON, o reconhecimento de palíndromos por AP e a computação da função sucessor em unário por MT.

2. FUNDAMENTAÇÃO TEÓRICA

2.1 Autômato Finito Determinístico (AFD)

Um AFD é definido formalmente como uma 5-tupla $M = (Q, \Sigma, \delta, q_0, F)$, onde:

- Q é um conjunto finito de estados;
- Σ é o alfabeto de entrada;
- $\delta: Q \times \Sigma \rightarrow Q$ é a função de transição;
- $q_0 \in Q$ é o estado inicial;
- $F \subseteq Q$ é o conjunto de estados de aceitação.

Para a linguagem $L1 = \{ w \in \{a,b\}^* \mid w \text{ termina com } ab \}$, construímos o AFD com $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{a,b\}$, $q_0 = q_0$, $F = \{q_2\}$ e a função de transição:

δ	a	b
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_0

2.2 Autômato de Pilha (AP) com aceitação por pilha vazia

Um AP é uma 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, \emptyset)$, onde:

- Γ é o alfabeto da pilha;
- $Z_0 \in \Gamma$ é o símbolo inicial da pilha;
- $\delta: Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow P(Q \times \Gamma^*)$ é a função de transição;
- O conjunto de estados de aceitação é vazio, pois a aceitação se dá quando a pilha se esvazia após o consumo total da entrada.

Para $L_2 = \{ a^n b^n \mid n \geq 1 \}$, a estratégia é: empilhar um marcador A para cada a lido; ao ler b, desempilhar um A; aceitar se a entrada terminar e a pilha estiver vazia.

2.3 Máquina de Turing (MT)

Uma MT é uma 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, onde:

- $\Gamma \supseteq \Sigma$ é o alfabeto da fita, incluindo o símbolo branco $_$;
- $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ é a função de transição;
- q_{accept} e q_{reject} são os estados de aceitação e rejeição, respectivamente.

Para $L_4 = \{ a^n b^n c^n \mid n \geq 1 \}$, adotamos uma estratégia de marcação: substituímos o primeiro a não marcado por X, o primeiro b não marcado por Y, o primeiro c não marcado por Z, e repetimos o processo. Se ao final restarem apenas marcas e brancos, a palavra é aceita.

3. DESCRIÇÃO DA IMPLEMENTAÇÃO

3.1 Estruturas de dados escolhidas

- AFD: Utilizamos `HashSet<string>` para os estados Q e para o conjunto de aceitação F; `HashSet<char>` para o alfabeto Σ ; um `Dictionary<(string, char), string>` para representar a função de transição δ ; e uma variável string para o estado inicial. O rastro dos estados percorridos é armazenado em uma `List<string>`.

- AP: A pilha foi implementada com `Stack<char>` (estrutura nativa do C#). As transições foram armazenadas em uma lista de objetos personalizados contendo: estado de origem, símbolo lido (ou null para ϵ), topo da pilha, estado de destino e string a ser empilhada. A aceitação ocorre exclusivamente quando a pilha está vazia e toda a entrada foi consumida.

- MT: A fita dinâmica foi modelada como `Dictionary<int, char>`, onde a chave é a posição e o valor é o caractere. O símbolo branco é representado por `'_'`. A função de transição é um `Dictionary<(string estado, char leitura), (string novo Estado, char`

escreve, char direção)>. Implementamos contador de passos com limite configurável para evitar loops infinitos.

3.2 Decisões de projeto

- AFD: Implementamos um método `ExibirDiagrama()` que imprime uma tabela textual das transições, conforme exigido. O desafio de carregar um AFD a partir de um arquivo JSON foi resolvido usando `System.Text.Json`, sem bibliotecas externas.

- AP: Para atender o desafio dos palíndromos (L3), implementamos uma busca recursiva que explora transições não-determinísticas (incluindo movimentos ϵ). A cada passo, o programa exibe a configuração instantânea (estado, conteúdo da pilha e resto da entrada).

- MT: A exibição passo a passo mostra o estado atual, a fita completa com o símbolo sob o cabeçote delimitado por [], e a posição. Implementamos uma segunda MT que calcula $n+1$ em unário, percorrendo a fita até o primeiro branco, escrevendo um 1 e retornando ao início.

3.3 Tratamento de erros e entradas inválidas

- AFD: Símbolos fora do alfabeto são rejeitados imediatamente. Se a função de transição não for total (falta uma transição para um par (estado, símbolo)), o programa também rejeita a cadeia.

- AP: Se durante a simulação não houver transição aplicável ou a pilha esvaziar antes do fim da entrada, a recursão retrocede e tenta outros caminhos (no caso não-determinístico). No final, se nenhum caminho levar ao esvaziamento da pilha com entrada esgotada, a palavra é rejeitada.

- MT: Transições indefinidas ou estados de rejeição interrompem a execução. O limite de passos evita execuções infinitas.

4. RESULTADOS E TESTES

4.1 AFD – $L1$ = palavras que terminam com "ab"

Cadeia	Rastro de estados	Resultado
ab	$q_0 \rightarrow q_1 \rightarrow q_2$	ACEITA
aab	$q_0 \rightarrow q_1 \rightarrow q_1 \rightarrow q_2$	ACEITA
bab	$q_0 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2$	ACEITA
ababab	$q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_1 \rightarrow q_2 \rightarrow q_1 \rightarrow q_2$	ACEITA
ba	$q_0 \rightarrow q_0 \rightarrow q_1$	REJEITA
ϵ	q_0	REJEITA
b	$q_0 \rightarrow q_0$	REJEITA

4.2 AP – $L2 = \{ a^n b^n \mid n \geq 1 \}$

Cadeia	Resultado
ab	ACEITA
aabb	ACEITA
aaabbb	ACEITA
aab	REJEITA
abb	REJEITA
ba	REJEITA
ϵ	REJEITA
abab	REJEITA

4.3 AP – Desafio L3 (palíndromos)

Cadeia	Resultado
a	ACEITA
aba	ACEITA
abba	ACEITA
ab	REJEITA
aab	REJEITA

4.4 MT – $L4 = \{ a^n b^n c^n \mid n \geq 1 \}$

Cadeia	Resultado
abc	ACEITA
aabbcc	ACEITA
aaabbbccc	ACEITA
aabbc	REJEITA
ab	REJEITA
abcabc	REJEITA
ϵ	REJEITA

4.5 MT – Desafio $f(n)=n+1$ (unário)

Entrada	Saída esperada	Resultado
1	11	ACEITA (fita: 11)
11	111	ACEITA (fita: 111)
111	1111	ACEITA (fita: 1111)
11111	111111	ACEITA (fita: 111111)

5. ANÁLISE COMPARATIVA

Característica	AFD	AP (pilha vazia)	MT
Classe de linguagem	Regulares (Tipo 3)	Livres de contexto (Tipo 2)	Recursivamente enumeráveis (Tipo 0)
Memória	Nenhuma (apenas estado)	Pilha LIFO	Fita infinita (leitura/escrita)
Exemplo reconhecido	Palavras terminadas com "ab"	$a^n b^n$	$a^n b^n c^n$

Poder computacional	Limitado	Médio	Turing-completo
------------------------	----------	-------	-----------------

A linguagem $L_2 = \{a^n b^n\}$ não é regular (pode ser provada pelo Lema do Bombeamento), portanto não pode ser reconhecida por um AFD, mas é livre de contexto, sendo aceita por um AP. Já $L_4 = \{a^n b^n c^n\}$ não é livre de contexto, exigindo uma MT, que graças à sua fita bidirecional e capacidade de marcação, consegue verificar a igualdade de três blocos.

6. QUESTÕES REFLEXIVAS – PARTE 1 (AFD)

6.1 Por que a linguagem L_1 é regular? Apresente a expressão regular equivalente.

A linguagem L_1 é regular porque pode ser reconhecida por um AFD, que possui um número finito de estados e nenhuma memória auxiliar. Além disso, toda linguagem regular pode ser representada por uma expressão regular. A expressão regular equivalente é $(a+b)^*ab$, onde $(a+b)^*$ representa qualquer sequência (inclusive vazia) de 'a' e 'b', seguida obrigatoriamente pelo sufixo "ab". Isso demonstra que L_1 pertence à classe das linguagens regulares.

6.2 Descreva formalmente a 5-tupla do AFD construído, justificando a escolha de cada estado.

A 5-tupla é: $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{a, b\}$, δ conforme tabela acima, $q_0 = q_0$, $F = \{q_2\}$.

- q_0 (inicial): representa que a última letra lida (se houver) não foi 'a' ou a cadeia ainda não terminou com "ab". Ao ler 'b', permanece em q_0 ; ao ler 'a', avança para q_1 .

- q_1 (meio): indica que o último símbolo lido foi 'a'. Aguarda um 'b' para completar o sufixo "ab". Se chegar outro 'a', permanece em q_1 ; se chegar 'b', vai para q_2 .

- q_2 (aceitação): indica que a última letra lida foi 'b' e a penúltima foi 'a', ou seja, o sufixo "ab" acabou de ser formado. Se a cadeia terminar aqui, é aceita. Se houver mais símbolos, o autômato continua a verificação a partir deste estado.

Esses três estados são suficientes porque o reconhecimento depende apenas do último e do penúltimo caractere, uma memória finita.

6.3 O que aconteceria se a função δ não fosse total? Como você tratou entradas inválidas?

Se δ não fosse total, haveria pares (estado, símbolo) sem transição definida, e o AFD não saberia como prosseguir, resultando na rejeição da cadeia. No código, tratamos isso de duas formas: (1) verificamos se o símbolo pertence ao alfabeto; (2) ao acessar o dicionário de transições, se a chave não existir, o método Aceitar retorna false imediatamente. Símbolos fora do alfabeto também causam rejeição imediata. Isso garante que a simulação é determinística e total no sentido de que todos os casos são cobertos (a rejeição é explícita).

7. QUESTÕES REFLEXIVAS – PARTE 2 (AP)

7.1 Por que L_2 não pode ser reconhecida por um AFD? Demonstre usando o Lema do Bombeamento.

$L_2 = \{ a^n b^n \mid n \geq 1 \}$ exige contar o número de 'a' e compará-lo com o número de 'b'. Um AFD possui memória finita (apenas um número fixo de estados), portanto não pode armazenar um valor arbitrário de n . Pelo Lema do Bombeamento para linguagens regulares: suponha que L_2 seja regular. Então existe um comprimento de bombeamento p . Escolha a palavra $w = a^p b^p$. Pelo lema, w pode ser decomposta em xyz , com $|xy| \leq p$, $|y| \geq 1$, e $xy^i z \in L_2$ para todo $i \geq 0$. Como $|xy| \leq p$, y consiste apenas de 'a's. Ao bombear y com $i=2$, obtemos $a^{p+|y|} b^p$, que tem mais 'a's do que 'b's, violando a condição de igualdade. Portanto, L_2 não é regular e não pode ser reconhecida por um AFD.

7.2 A aceitação por pilha vazia e por estado final são equivalentes? Demonstre conceitualmente.

Sim, são equivalentes. Dado um AP que aceita por estado final, podemos construir um AP que aceita por pilha vazia: adiciona-se um novo símbolo de fundo de pilha (X_0) e um novo estado que, ao atingir um estado final, entra em um loop que desempilha todos os símbolos até que reste apenas X_0 , que é então desempilhado, esvaziando a pilha. Reciprocamente, um AP que aceita por pilha vazia pode ser convertido para aceitação por estado final: adiciona-se um novo estado final e uma transição que, quando a pilha estiver vazia (apenas o marcador de fundo Z_0), move para esse estado final. Ambos reconhecem exatamente a classe das linguagens livres de contexto.

7.3 Explique o papel do símbolo Z_0 na sua implementação.

Z_0 (implementado como 'Z') é o símbolo inicial da pilha e atua como um marcador de fundo. Durante a simulação, enquanto empilhamos e desempilhamos símbolos (como 'A' para a), Z_0 permanece no fundo. Quando toda a entrada é consumida e a

pilha contém apenas Z0, executamos uma transição λ que desempilha Z0, deixando a pilha completamente vazia, o que dispara a condição de aceitação. Sem Z0, não seria possível detectar o momento em que a pilha está "vazia" de símbolos significativos, tornando a aceitação por pilha vazia inviável.

8. QUESTÕES REFLEXIVAS – PARTE 3 (MT)

8.1 Por que L4 não pode ser reconhecida por um Autômato de Pilha? Qual propriedade da MT possibilita o seu reconhecimento?

$L4 = \{ a^n b^n c^n \mid n \geq 1 \}$ exige comparar três contagens (a, b, c). Um AP possui apenas uma pilha LIFO, que consegue emparelhar dois blocos (como a e b), mas ao terminar a verificação de b, a pilha está vazia e não há como verificar a contagem de c. A MT, por outro lado, tem uma fita bidirecional com capacidade de leitura e escrita, permitindo marcar os símbolos já processados (por exemplo, substituir a por X, b por Y, c por Z). Essa capacidade de "marcação" e de movimentação livre (esquerda/direita) permite à MT verificar quantidades iguais de três símbolos, o que é impossível para um AP.

8.2 Quantos estados foram necessários para a MT de L4? Descreva a estratégia de "marcação" adotada.

Foram necessários 7 estados: q0, q1, q2, q3, q4, qaccept, qreject. A estratégia de marcação é a seguinte:

- q0: procura o primeiro 'a' não marcado. Encontra 'a' e substitui por X, vai para q1. Se encontrar 'Y' (significa que todos os 'a' já foram marcados), vai para q4 para verificação final.
- q1: move para a direita, ignorando 'a' e 'Y', até encontrar o primeiro 'b' não marcado. Substitui por Y e vai para q2.
- q2: move para a direita, ignorando 'b' e 'Z', até encontrar o primeiro 'c' não marcado. Substitui por Z e vai para q3.
- q3: move para a esquerda até encontrar um X (início do próximo ciclo) e volta para q0.
- q4: percorre a fita verificando se só restam Y, Z e branco; se sim, vai para qaccept; senão, para qreject.

Esse processo garante que a cada ciclo um 'a', um 'b' e um 'c' são marcados, e ao final todos estarão marcados se as quantidades forem iguais.

8.3 Um computador moderno é mais poderoso que uma Máquina de Turing? Justifique à luz da Tese de Church-Turing.

Não. De acordo com a tese de Church-Turing, qualquer função que pode ser computada por um algoritmo (em qualquer computador real) também pode ser

computada por uma Máquina de Turing. Computadores modernos possuem memória finita e são, na prática, equivalentes a Máquinas de Turing com fita limitada. Embora sejam muito mais rápidos e eficientes, eles não podem resolver problemas que são indecidíveis para uma MT (como o problema da parada). Portanto, o poder computacional é o mesmo: ambos são Turing-completos. A diferença está apenas na eficiência e na praticidade, não no poder teórico.

9. CONCLUSÃO

A implementação das três máquinas permitiu compreender na prática os conceitos teóricos de computabilidade e a hierarquia de Chomsky. O AFD mostrou-se simples mas limitado; o AP, com sua pilha, resolveu problemas de contagem ($a^n b^n$) e ainda, com não-determinismo, reconheceu palíndromos; a MT demonstrou ser um modelo universal, capaz de reconhecer $a^n b^n c^n$ e até computar funções aritméticas simples. As principais dificuldades encontradas foram a correta implementação dos movimentos lambda no AP (especialmente para o não-determinismo) e a exibição dinâmica da fita na MT. O uso de recursão e cópia da pilha foi essencial para explorar múltiplos caminhos. O trabalho reforçou a importância de traduzir formalismos matemáticos diretamente em código, bem como a necessidade de documentação e testes sistemáticos.

10. REFERÊNCIAS

SIPSER, Michael. Introduction to the Theory of Computation. 3. ed. Boston: Cengage Learning, 2013.

HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. Introdução à Teoria de Autômatos, Linguagens e Computação. 2. ed. Rio de Janeiro: Campus/Elsevier, 2003.

MENEZES, Paulo Blauth. Linguagens Formais e Autômatos. 6. ed. Porto Alegre: Bookman, 2010.

LEWIS, Harry R.; PAPADIMITRIOU, Christos H. Elements of the Theory of Computation. 2. ed. Upper Saddle River: Prentice Hall, 1998.